RASPBERRY PI TELEGRAM BOT

Light corporate cyberpunk field manual - English edition

Build a reliable Telegram bot on Raspberry Pi with Python, SQLite, systemd, BotFather configuration and optional local AI via Ollama.

> **COPY-PASTE FIRST // Most pages contain terminal blocks designed for direct use. Replace placeholders like BOT_TOKEN_HERE and raspberrypi.local before running.**

Stack	Recommended baseline
OS	Raspberry Pi OS Lite, 64-bit
Language	Python 3 with a virtual environment
Bot library	python-telegram-bot v22.x API style
Runtime	systemd service with automatic restart
Storage	SQLite for notes, logs or config snapshots
Optional AI	Ollama on Pi 4/5 with small quantized models

[FIELD MANUAL]

00 // Mission Brief

This guide creates a general-purpose Telegram bot on a Raspberry Pi. It does not copy any existing private bot. The sample code is intentionally generic: it can answer status checks, store short notes in SQLite, show recent entries, export a backup file and run as a service after reboot.

- Use BotFather to create a bot account and token.
- Prepare Raspberry Pi OS, SSH access and a Python virtual environment.
- Run a minimal bot locally, then install it as a systemd service.
- Add SQLite storage for notes or small command logs.
- Optionally attach a small local model through Ollama.

> **SECURITY NOTE // Never paste your bot token into public chats, screenshots, GitHub issues or support forums. Treat it like a password. The Telegram bot creation flow follows Telegram official BotFather docs [S1].**

[NODE SELECTION]

01 // Raspberry Pi Hardware Map

For a normal Telegram bot, even older boards are enough. Local AI is the part that changes the hardware requirement. A bot that only receives commands, writes SQLite rows and sends Telegram replies is lightweight. A bot that runs a local language model is not.

Board	Profile	Bot + AI guidance
Pi 3 / Pi 3 B+	1GB RAM, quad-core Cortex-A53 class	Fine for simple Telegram bots, cron jobs, sensors and SQLite. Not recommended for local LLMs.
Pi 4 - 2GB	Entry Pi 4	Good for simple bots. Skip local AI or use remote API.
Pi 4 - 4GB	Balanced older board	Good for bots and light services. Tiny local models only; expect slow CPU-only replies.
Pi 4 - 8GB	Best Pi 4 for local experiments	Can try 0.5B-1.5B quantized models. 3B is possible but not fluid.
Pi 5 - 4GB	Much faster CPU than Pi 4	Strong bot host. Small local models only; 0.5B-1.5B recommended.
Pi 5 - 8GB	Recommended modern baseline	Good for Telegram + SQLite + RSS + small local AI. 1.5B-3B quantized feels most reasonable.
Pi 5 - 16GB	Best Raspberry option	More memory headroom. 3B-4B comfortable; 7B can run but will not feel truly fluid on CPU.

Raspberry Pi product pages list Pi 3 B+ with 1GB LPDDR2 RAM, Pi 4 variants up to 8GB, and Pi 5 variants up to 16GB. Pi 5 uses a 2.4GHz Cortex-A76 class CPU and is described as up to three times faster than the previous generation [S3-S5].

[OPTIONAL OLLAMA]

02 // Local AI Sizing

The bot does not need a local model. Add one only when you want offline-ish summarizing, note cleanup or local chat. On Raspberry Pi, think in model parameter size, quantization and expected latency. Do not expect desktop-GPU behavior.

Host	Relatively smooth range	Notes
Pi 3	No local LLM	Use rules, commands or a remote API. 0.5B is experimental and usually not worth it.
Pi 4 4GB	0.5B Q4	Basic cleanup or short replies. Keep prompts short.
Pi 4 8GB	0.5B-1.5B Q4	Usable for small assistant behavior. 3B only if latency is acceptable.
Pi 5 4GB	0.5B-1.5B Q4	Good for short local actions; keep context small.
Pi 5 8GB	1.5B-3B Q4/Q5	Best practical Raspberry range for a responsive local bot.
Pi 5 16GB	3B-4B Q4/Q5; 7B Q4 possible	7B is memory-possible but CPU-limited. Prefer 3B-4B for smoother chat.

Copy-paste: model smoke tests

```
# Official small baseline
ollama run qwen2.5:0.5b
ollama run qwen2.5:1.5b
ollama run qwen2.5:3b

# Huihui examples - check current tags on ollama.com/huihui_ai
ollama run huihui_ai/qwen3-abliterated:0.6b
ollama run huihui_ai/qwen3-abliterated:1.7b
ollama run huihui_ai/qwen3-abliterated:4b

# Heavier; Pi 5 8/16GB only, expect slower CPU replies
ollama run huihui_ai/phi4-mini-abliterated
```

Ollama Linux docs use curl -fsSL https://ollama.com/install.sh | sh. Qwen2.5 has sizes from 0.5B upward; Huihui model tags include small abliterated Qwen-family options. Abliterated models have reduced safety filtering, so use them only in private, controlled deployments [S7-S9].

[TELEGRAM CONTROL PLANE]

03 // BotFather Configuration

Open Telegram, search for @BotFather, then create a bot. The token is the credential your Raspberry Pi uses to talk to Telegram. Store it only in .env on the Pi.

Copy-paste: create the bot

```
/newbot

# BotFather asks for a display name:
Raspi Node Bot

# BotFather asks for a username. It must end in bot:
raspi_node_demo_bot
```

Copy-paste: bot profile and command menu

```
/setdescription
Raspberry Pi automation node for notes, status checks and small commands.

/setabouttext
Self-hosted Raspberry Pi Telegram bot.

/setcommands
start - boot message
status - service health
note - save a note
last - show recent notes
backup - export notes
help - command list
```

GROUPS // If the bot should read every message in a group, ask BotFather for /setprivacy and disable privacy mode. For private one-to-one bots, leave privacy as default.

[BOOTSTRAP]

04 // Raspberry Pi OS + SSH

Use Raspberry Pi Imager on your PC. Recommended image: Raspberry Pi OS Lite 64-bit. In Imager advanced settings, set hostname, username, password, Wi-Fi and enable SSH. This gives you a headless node you can reach from PuTTY or Windows Terminal.

Copy-paste: first SSH login

```
# From Windows PowerShell or a terminal on your PC:
ssh pi@raspberrypi.local

# If you set a custom hostname in Raspberry Pi Imager:
ssh pi@raspi-bot.local
```

Copy-paste: confirm the node

```
# On the Pi after login:
hostname
ip addr show
uname -a
```

> **Raspberry Pi documentation describes SSH as the standard way to access a remote terminal session on the local network [S6].**

[SYSTEM PREP]

05 // Base System Install

Update the board, install Python tooling, create a project folder and build a virtual environment. The commands below assume the normal user is pi. If your username is different, the systemd service later must use your path.

Copy-paste: system and Python setup

```
sudo apt update
sudo apt full-upgrade -y
sudo apt install -y python3-venv python3-pip sqlite3 git curl nano

mkdir -p ~/telegram-bot
cd ~/telegram-bot
python3 -m venv venv
./venv/bin/python -m pip install --upgrade pip
./venv/bin/pip install python-telegram-bot==22.8
```

Copy-paste: .env token file

```
cd ~/telegram-bot
cat > .env <<'EOF'
BOT_TOKEN=PASTE_BOTFATHER_TOKEN_HERE
BOT_NAME=raspi-node
EOF
chmod 600 .env
```

[BOT CORE]

06 // General Bot Code - Part 1

This is a generic starter bot. It stores notes in SQLite, replies immediately, has a status command and can show recent notes. It is intentionally small enough to understand and extend.

Copy-paste: bot.py part 1

```
cd ~/telegram-bot
cat > bot.py <<'PY'
import os
import re
import sqlite3
from datetime import datetime
from pathlib import Path
from zoneinfo import ZoneInfo

from telegram import Update
from telegram.ext import (
    Application,
    CommandHandler,
    MessageHandler,
    ContextTypes,
    filters,
)

BASE = Path(__file__).resolve().parent
DB = BASE / "bot.db"
ENV = BASE / ".env"
TZ = ZoneInfo("Europe/Berlin")


def load_env():
    if ENV.exists():
        for line in ENV.read_text().splitlines():
            line = line.strip()
            if not line or line.startswith("#") or "=" not in line:
                continue
            key, value = line.split("=", 1)
            os.environ.setdefault(key.strip(), value.strip())


def now_text():
    return datetime.now(TZ).isoformat(timespec="seconds")


def clean_text(text):
    text = str(text or "").strip()
    text = re.sub(r"\s+", " ", text)
    return text
PY
```

[BOT CORE]

07 // General Bot Code - Part 2

Copy-paste: bot.py part 2

```
cd ~/telegram-bot
cat >> bot.py <<'PY'


def db():
    con = sqlite3.connect(DB)
    con.row_factory = sqlite3.Row
    return con


def init_db():
    con = db()
    con.execute(
        """
        CREATE TABLE IF NOT EXISTS notes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            created_at TEXT NOT NULL,
            text TEXT NOT NULL
        )
        """
    )
    con.commit()
    con.close()


def save_note(text):
    text = clean_text(text)
    if not text:
        raise ValueError("empty note")

    con = db()
    cur = con.cursor()
    cur.execute(
        "INSERT INTO notes (created_at, text) VALUES (?, ?)",
        (now_text(), text),
    )
    note_id = cur.lastrowid
    con.commit()
    con.close()
    return note_id


def get_last(limit=10):
    con = db()
    rows = con.execute(
        "SELECT id, created_at, text FROM notes ORDER BY id DESC LIMIT ?",
        (limit,),
    ).fetchall()
    con.close()
    return list(reversed(rows))
PY
```

[BOT CORE]

08 // General Bot Code - Part 3

Copy-paste: handlers and status

```
cd ~/telegram-bot
cat >> bot.py <<'PY'


async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text(
        "[RASPI.NODE] online\n"
        "Send any text to store it.\n"
        "Commands: /status /last /backup /help"
    )


async def status(update: Update, context: ContextTypes.DEFAULT_TYPE):
    con = db()
    total = con.execute("SELECT COUNT(*) FROM notes").fetchone()[0]
    con.close()
    await update.message.reply_text(
        f"[STATUS] online\nnotes: {total}\ntime: {now_text()}"
    )


async def last(update: Update, context: ContextTypes.DEFAULT_TYPE):
    limit = 10
    if context.args:
        try:
            limit = max(1, min(50, int(context.args[0])))
        except ValueError:
            limit = 10

    rows = get_last(limit)
    if not rows:
        await update.message.reply_text("[VAULT] empty")
        return

    lines = ["[LAST.NOTES]"]
    for row in rows:
        lines.append(f"- #{row['id']} {row['text']}")
    await update.message.reply_text("\n".join(lines))
PY
```

[BOT CORE]

09 // General Bot Code - Part 4

Copy-paste: backup, intake and main

```
cd ~/telegram-bot
cat >> bot.py <<'PY'


async def backup(update: Update, context: ContextTypes.DEFAULT_TYPE):
    rows = get_last(100000)
    out = BASE / "notes-backup.txt"

    lines = []
    for row in rows:
        lines.append(f"#{row['id']} {row['created_at']} - {row['text']}")

    out.write_text("\n".join(lines) + "\n", encoding="utf-8")
    await update.message.reply_document(out.open("rb"), filename=out.name)


async def handle_text(update: Update, context: ContextTypes.DEFAULT_TYPE):
    text = clean_text(update.message.text)
    if not text:
        return

    try:
        note_id = save_note(text)
        await update.message.reply_text(f"[LOGGED]\n- #{note_id} {text}")
    except Exception as exc:
        await update.message.reply_text(f"[ERROR] note not stored: {exc}")


def main():
    load_env()
    init_db()

    token = os.environ.get("BOT_TOKEN", "").strip()
    if not token:
        raise RuntimeError("BOT_TOKEN missing in .env")

    app = Application.builder().token(token).build()
    app.add_handler(CommandHandler("start", start))
    app.add_handler(CommandHandler("help", start))
    app.add_handler(CommandHandler("status", status))
    app.add_handler(CommandHandler("last", last))
    app.add_handler(CommandHandler("backup", backup))
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND,
                                   handle_text))

    app.run_polling(drop_pending_updates=True,
                    allowed_updates=Update.ALL_TYPES)


if __name__ == "__main__":
    main()
PY
```

[SMOKE TEST]

10 // First Local Test

Before installing a service, run the bot manually. Keep this terminal open. Open Telegram, send /start to your bot, then send a normal text message.

Copy-paste: run manually

```
cd ~/telegram-bot
set -a
. ./.env
set +a
./venv/bin/python bot.py
```

Telegram smoke test

```
/start
[RASPI.NODE] online

hello from raspberry
[LOGGED]
- #1 hello from raspberry

/status
[STATUS] online
notes: 1
```

STOP MANUAL MODE // Press Ctrl+C before installing the systemd service. Otherwise the service and manual process may fight over the same Telegram updates.

[AUTOSTART]

11 // systemd Service

systemd keeps the bot running after reboot and restarts it if the Python process crashes. The systemd service manual defines unit options such as ExecStart, Restart and WorkingDirectory [S10].

Copy-paste: create and start service

```
sudo tee /etc/systemd/system/telegram-bot.service > /dev/null <<'EOF'
[Unit]
Description=Raspberry Pi Telegram Bot
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/telegram-bot
EnvironmentFile=/home/pi/telegram-bot/.env
ExecStart=/home/pi/telegram-bot/venv/bin/python /home/pi/telegram-bot/bot.py
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now telegram-bot
```

Copy-paste: service diagnostics

```
systemctl status telegram-bot --no-pager -l
journalctl -u telegram-bot -n 80 --no-pager
journalctl -u telegram-bot -f
```

[OPERATIONS]

12 // Maintenance Commands

These are the commands you will use most often after the first install.

Copy-paste: operations block

```
# Restart after editing bot.py
sudo systemctl restart telegram-bot

# Stop the bot
sudo systemctl stop telegram-bot

# Start again
sudo systemctl start telegram-bot

# Show recent logs
journalctl -u telegram-bot -n 120 --no-pager

# Check Python syntax before restarting
cd ~/telegram-bot
./venv/bin/python -m py_compile bot.py
```

Copy-paste: backup project folder

```
# Manual project backup
cd ~
tar -czf telegram-bot-backup-$(date +%Y%m%d-%H%M%S).tar.gz telegram-bot
ls -lh telegram-bot-backup-*.tar.gz | tail -n 5
```

[LOCAL AI MODULE]

13 // Optional: Add Ollama

Use this only after the plain bot is stable. The bot remains useful without local AI. On Raspberry Pi, keep models small and prompts short.

Copy-paste: Ollama install and tiny model

```
# Install Ollama on Linux
curl -fsSL https://ollama.com/install.sh | sh

# Start and enable service
sudo systemctl enable --now ollama
systemctl status ollama --no-pager -l

# Pull a small model first
ollama pull qwen2.5:0.5b
ollama run qwen2.5:0.5b
```

Copy-paste: local model candidates

```
# Better Pi 5 tests, depending on RAM
ollama pull qwen2.5:1.5b
ollama pull qwen2.5:3b

# Huihui examples; check exact current tags online first
ollama pull huihui_ai/qwen3-abliterated:0.6b
ollama pull huihui_ai/qwen3-abliterated:1.7b
ollama pull huihui_ai/qwen3-abliterated:4b
```

[OPTIONAL CODE]

14 // Python Helper for Local AI

Add this helper to bot.py only after the normal bot works. It calls Ollama locally through HTTP. If Ollama is down, the bot returns a clean error instead of freezing forever.

Copy-paste: Ollama helper

```
# Add near the top of bot.py:
import json
import urllib.request


def ask_ollama(prompt, model="qwen2.5:0.5b", timeout=30):
    payload = json.dumps({
        "model": model,
        "prompt": prompt,
        "stream": False,
        "options": {"temperature": 0.2, "num_predict": 160},
    }).encode("utf-8")

    req = urllib.request.Request(
        "http://127.0.0.1:11434/api/generate",
        data=payload,
        headers={"Content-Type": "application/json"},
        method="POST",
    )

    with urllib.request.urlopen(req, timeout=timeout) as resp:
        data = json.loads(resp.read().decode("utf-8"))
        return data.get("response", "").strip()
```

Copy-paste: /ai command skeleton

```
# Add this handler to bot.py:
async def ai(update: Update, context: ContextTypes.DEFAULT_TYPE):
    prompt = " ".join(context.args).strip()
    if not prompt:
        await update.message.reply_text("Usage: /ai your question")
        return

    try:
        answer = await asyncio.to_thread(ask_ollama, prompt)
    except Exception as exc:
        answer = f"AI module offline: {exc}"

    await update.message.reply_text(answer[:3500])

# Also add in main():
# app.add_handler(CommandHandler("ai", ai))
```

When using asyncio.to_thread, also add import asyncio near the top of bot.py.

[FAILSAFE]

15 // Security + Troubleshooting

Most bot failures are not complex: wrong token, two processes using the same token, service path mismatch, missing .env, or a blocked network path to Telegram.

Symptom	Fast fix
Bot does not reply	Check service logs; confirm BOT_TOKEN; confirm only one process uses the token.
Conflict / getUpdates error	Stop manual bot process, restart systemd service.
Works manually, not as service	Check User, WorkingDirectory, ExecStart and EnvironmentFile paths.
Notes not stored	Run sqlite3 bot.db '.schema' and check file permissions.
Very slow replies	Remove local AI from message intake. Use AI only behind a command.
Telegram delays	Check internet, DNS, IPv6 issues and journalctl output.

Copy-paste: diagnostic block

```
# Check for duplicate bot processes
ps aux | grep bot.py | grep -v grep

# Kill manual stuck process if needed
pkill -f '/home/pi/telegram-bot/bot.py'
sudo systemctl restart telegram-bot

# Inspect database
cd ~/telegram-bot
sqlite3 bot.db '.tables'
sqlite3 bot.db 'select * from notes order by id desc limit 5;'
```

[FAST LANE]

16 // Copy-Paste Command Sheet

Minimal full install path, assuming the bot code from this PDF is already written to ~/telegram-bot/bot.py.

Copy-paste: compact install

```
# 1. System packages
sudo apt update
sudo apt install -y python3-venv python3-pip sqlite3 curl

# 2. Python environment
mkdir -p ~/telegram-bot
cd ~/telegram-bot
python3 -m venv venv
./venv/bin/pip install --upgrade pip
./venv/bin/pip install python-telegram-bot==22.8

# 3. Token file
cat > .env <<'EOF'
BOT_TOKEN=PASTE_BOTFATHER_TOKEN_HERE
EOF
chmod 600 .env

# 4. Test syntax
./venv/bin/python -m py_compile bot.py

# 5. Manual run
set -a; . ./.env; set +a
./venv/bin/python bot.py
```

[REFERENCE GRID]

17 // Sources

Sources used for platform facts, setup flow and current package/model references. URLs are written out so the PDF remains useful outside chat.

ID	Source	URL
S1	Telegram Bot Tutorial / BotFather	https://core.telegram.org/bots/tutorial
S2	python-telegram-bot documentation	https://docs.python-telegram-bot.org/
S3	Raspberry Pi 3 Model B+ product page	https://www.raspberrypi.com/products/raspberry-pi-3-model-b-plus/
S4	Raspberry Pi 4 Model B product page	https://www.raspberrypi.com/products/raspberry-pi-4-model-b/
S5	Raspberry Pi 5 product page	https://www.raspberrypi.com/products/raspberry-pi-5/
S6	Raspberry Pi SSH documentation	https://www.raspberrypi.com/documentation/usage/network/ssh/
S7	Ollama Linux install documentation	https://docs.ollama.com/linux
S8	Ollama Qwen2.5 library	https://ollama.com/library/qwen2.5
S9	Ollama Huihui profile	https://ollama.com/huihui_ai
S10	systemd service manual	https://www.freedesktop.org/software/systemd/man/latest/systemd.service.html

FINAL RULE // Keep the normal bot fast. Put slow AI behind explicit commands like /ai or /brief. Never run local models inside every message handler unless you accept delays.